Design, Implementation, and Evaluation of CNNs for Robust Image Classification on CIFAR-10

Shashikant Chaudhary
The University of Adelaide

Abstract

Convolutional Neural Networks are considered to be very efficient for image classification tasks. This work discusses a thorough analysis of the various CNN architectures on the CIFAR-10 dataset, which comprises our own CNN model, ResNet-18, MobileNetV3, and AlexNet. These models were implemented, and the best parameter was obtained using hyperparameter tuning done using Optuna. Later we presented how the performance of these models is affected by data augmentation. A performance evaluation was done based on accuracy, generalization capability, and computational efficiency. Among those, our custom CNN model performed best with a test accuracy of 86.96%, outperforming other architectures without sacrificing computational efficiency. Data augmentation greatly improved model generalization, significantly relieving the overfitting problem for more complex models. These results signify that our custom CNN model generalizes well and offers a balance between accuracy and computational efficiency.

1 Introduction and Related Work

Convolutional Neural Networks (CNNs) have revolutionized the field of computer vision, driving state-of-the-art results in image classification, object detection, and segmentation. Specifically, image classification has been a big success with deep learning models; there have been a number of architectures designed and benchmarked on CIFAR-10 datasets. CIFAR-10 is a low resolution image based 10 classes classification problem but it remains a good reference to determine the computational complexity of various CNN architectures under different constraints on the model and input preprocessing.

The pioneering work of Kritzhevsky et al on AlexNet [1] was one of the first leveraging deep CNNs on large scale image classification which has achieved significant performance when compared with previous methods. VGG [2] followed this by creating deep CNN architectures with small convolutional filters to demonstrate how depth in CNN models enables the model to learn more data. ResNet [3] introduced residual connections, overcoming the vanishing gradient problem, amenable to learning much deeper networks that achieve superior performance on many benchmarks. Howard et al. [4] continued in this vein, creating mobile and embedded suitable models by using depthwise separable convolutions, while maintaining accuracy.

While these established architectures are able to perform well on image classification tasks, our study aims to fill this gap by implementing and comparing a custom CNN model trained for CIFAR-10 with ResNet18, MobileNet, and AlexNet. We also employ hyperparameter tuning with Optuna to maximize model performance with resource restraints and study the effects of data augmentation on model accuracy and generalization. Lastly, this work sheds light on the trade offs between model complexity, accuracy, and computational cost towards the creation of suitable CNN models for image classification in the resource constrained environment.

2 Dataset

The CIFAR(Canadian Institute For Advanced Research) dataset is a popular benchmarking dataset for classification tasks, it contains small images of 32×32 pixels which are categorized into 10 classes(CIFAR-10). The images are categorized into objects like airplanes, automobiles, trucks, etc [11].

The dataset contains 50,000 images for training and 10,000 images for testing, Each category has 6,000 images which helps models distinguish between these 10 objects.The Figure 1 shows sample images from different classes in the CIFAR-10 dataset.

3 Data Preprocessing and Augmentation

We performed data preprocessing and augmentation techniques to generalise the model well and not overfit.

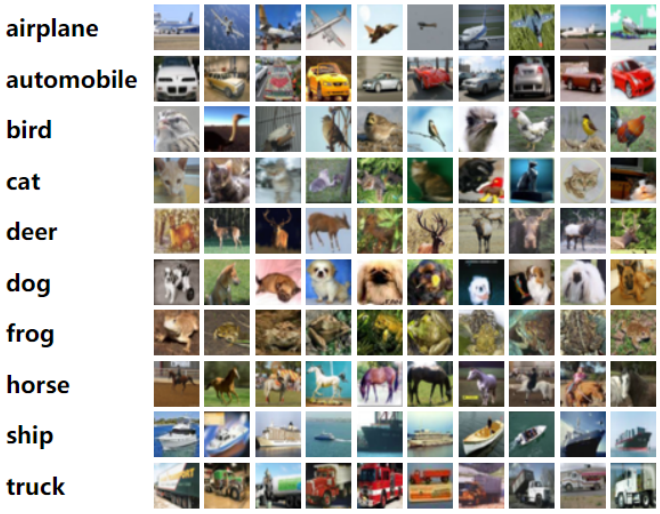


Figure 1: Sample images from the CIFAR-10 dataset across different classes.

3.1 Preprocessing

We used CIFAR-10 as a dataset, where all images were normalized as:

$$\text{Normalized Image} = \frac{\text{Image} - \mu}{\sigma}$$

with μ being the mean and σ being standard deviation vectors for each color channel.

3.2 Data Augmentation

The following augmentation techniques were used by us:

- **Random Horizontal Flip**: Images were randomly flipped horizontally with a probability of 0.5.
- **Random Crop**: Images were padded with 4 pixels on each side, and then we took a random crop to 32×32.
- **Random Rotation**: Images were rotated randomly with an angle between $[-15°, 15°]$.

To increase diversity in the training set, data augmentations let the models learn invariant features and create robustness.

4 Network Architectures

We implemented several architectures in our experiment, including a custom CNN model specifically designed to overcome overfitting problems and generalize well on classification problems. We implemented other popular architectures such as ResNet-18, MobileNetV3, and AlexNet and made some minor changes tailored for the Cifar 10 dataset.

4.1 Our CNN Model

Our Custom CNN model is specifically designed for image classification and implements several advanced features to enhance feature extraction, perform regularization, and improve gradient flow.

4.1.1 Convolution Operation

Mathematically the convolution operation can be defined as:

$$S(i,j) = \sum_m \sum_n X(i-m, j-n) \cdot W(m,n) \qquad (1)$$

where X is the input feature map, W is the convolution kernel, and S is the output feature map at position (i,j).

4.1.2 Mish Activation Function

We implemented Mish Activation which can be defined as follows:

$$\text{Mish}(x) = x \cdot \tanh\left(\ln\left(1 + e^x\right)\right) \qquad (2)$$

Mish is a smooth, non-monotonic function, unlike Relu changes sharply at 0. It was introduced in 2019 by Diganta Misra [5]. This activation function retains a small negative value and its smooth gradient often helps with training stability which is not in the case of ReLU. Difference between Mish and ReLU can be seen in the Figure 2.

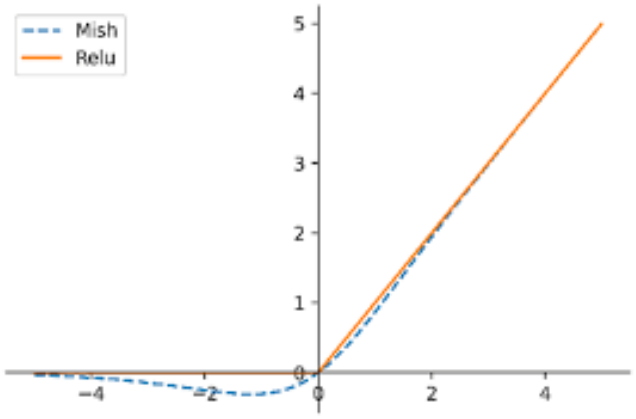


Figure 2: Comparison of Mish and ReLU activation function.

4.1.3 Network Architecture Overview

Our architecture contains a total of three convolutional blocks and then followed by the fully connected layers. Each block is designed in such a way that it can capture the features at different levels of abstraction.

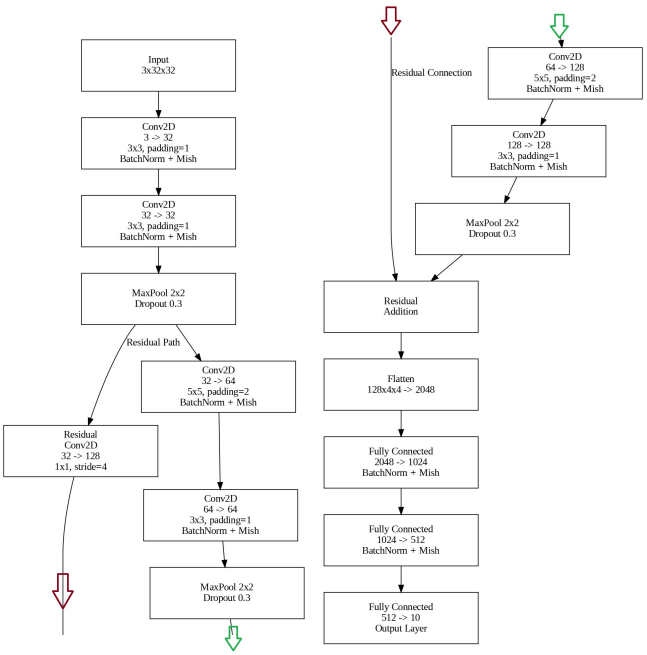


Figure 3: Architecture of the Our Custom CNN Model.

4.1.4 Residual Connection

We introduced a residual connection right after the first pooling layer and the rejoined the network after the third pooling layer. In order to match the dimensions a 1×1 convolution a 1×1 convolution with a stride of 4 has been used in the residual path to downsample and increase the number of channels from 32 to 128. Mathematically, the residual connection is implemented as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{R}(\mathbf{x}) \tag{3}$$

where $\mathbf{x}$ is the input after the first pooling layer, $\mathcal{F}(\mathbf{x})$ represents the transformations in the main path (second and third convolutional blocks), and $\mathbf{R}(\mathbf{x})$ is the residual mapping implemented using the 1×1 convolution [3].

4.1.5 Batch Normalization and Dropout

We introduced Batch Normalisation [6] after every convolutional layer, this stabilizes the training by normalizing inputs. Also, dropout [7] has been applied with the value of 0.3 after every pooling so that overfitting can be prevented.

4.1.6 Fully Connected Layers

Later the fully connected layer is applied and it consists of:

- A layer with 1024 Neurons, and then followed by batch normalization with Mish activation function.
- Another layer with 512 Neurons, and then followed by batch normalization with Mish activation function.
- An output layer with 10 units (one for each class).

4.1.7 Loss Function

Since it is a classification problem, we have used cross entropy loss which can be defined as follows:

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c) \tag{4}$$

where C is the number of classes, y_c is the true label, and $\hat{y}_c$ is the predicted probability.

4.2 ResNet-18

The ResNet-18 architecture contains 18 layer, in order to fit to our problem we modified the initial convolutional layer so that it can accept the input of 32×32.

ResNet-18 architecture introduces residual connection which allows it to train deeper networks by removing the problem of vanishing gradient. The residual connection in these layers is is defined in the equation(3).The residual connection is represented by adding the function $\mathcal{F}(\mathbf{x})$ and the residual $\mathbf{R}(\mathbf{x})$.

4.3 MobileNetV3

The MobileNetV3 architecture improves the efficiency by introducing depthwise separable convolutions and combines it with with squeeze-and-excitation module for neural architecture search. Mathematically, the standard Consultation Cost can represented as follows:

$$\begin{aligned} &\text{Standard Convolution Cost} \\ &= D_K^2 \cdot M \cdot N \cdot D_F^2 \end{aligned} \tag{5}$$

On the other hand, the depth-wise separable convolution along with additional efficiency from squeeze-and-excitation modules can be written as follows:

$$\begin{aligned} &\text{MobileNetV3 Convolution Cost} \\ &= D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2 + \delta \cdot M \cdot D_F^2 \end{aligned} \tag{6}$$

where D_K is the kernel size, M is the number of input channels, N is the number of output channels, D_F is the spatial dimension of the feature map, and δ represents the additional cost from squeeze-and-excitation modules [4].

4.4 AlexNet

AlexNet is an early CNN architecture made by Alex Krizhevsky along with Ilya Sutskever and Geoffrey Hinton and was trained on the ImageNet dataset. This architecture contains five convolutional layers and then is followed by 3 fully connected layers [1].

5 Hyperparameter Optimization using Optuna framework

To perform hyperparameter tuning we have used Optuna framewrok [9], and focused on the following parameters:

- **Learning Rate (η)**: Explored in the range of $[1 \times 10^{-4}, 1 \times 10^{-1}]$.
- **Optimizer**: Adam and SGD Optimizer with momentum.
- **Momentum**: Explored in the range of 0.5 to 0.99 for the SGD optimizer.
- **Weight Decay (λ)**: Explored in the range of $[1 \times 10^{-5}, 1 \times 10^{-2}]$.

Based on the availability of resources, we limited the trial to 10 and was trained the model for 15 epochs per trial. We also enabled the pruning so that unfavorable can be stopped in its early stage.

6 Implementation Details

6.1 Frameworks and Libraries

We implemented all the experiments using PyTorch Framework and used torch vision for handling the dataset. We also used Optuna Framework for the hyperparameter tuning.

6.2 Hardware

All the experiments were performed on the machine with the following configurations:

- CPU: 13th Gen Intel Core™ i7-13700HX
- GPU: NVIDIA GeForce RTX 4060 Laptop(8GB)
- RAM: 16 GB

7 Code

The code of the experiments performed in this experiment can be found at: https://github.com/chaudharysk/DeeeLearningA2.

8 Experiments and Results

We implemented the architecture with data augmentation and without augmentation in order to understand the impact of augmentation on model performance. This helped us to compare how augmentation influences the training and generalizability of the model.

8.1 Training Setup

- **Batch Size**: 128
- **Number of Epochs**: 15
- **Learning Rate**: Determined via hyperparameter tuning.
- **Optimizers**: SGD with momentum and Adam
- **Loss Function**: Cross-entropy loss
- **Learning Rate Scheduler**: StepLR, the step size is set to 10 so that it reduces the learning rate to half after the 10 epochs.

8.2 Best Hyperparameters for Each Model

We found the hyperparameters for each model via Optuna hyperparameter tuning and best parameters of each model is summmarised in Table 1.

Model	Augmentation	Optimizer	Learning Rate	Weight Decay
AlexNet	Yes	Adam	1.96×10^{-4}	1.13×10^{-3}
	No	Adam	1.96×10^{-4}	1.13×10^{-3}
MobileNetV3	Yes	Adam	7.48×10^{-4}	6.85×10^{-4}
	No	Adam	7.48×10^{-4}	6.85×10^{-4}
Custom CNN	Yes	Adam	3.06×10^{-3}	1.27×10^{-5}
	No	Adam	7.48×10^{-4}	6.85×10^{-4}
ResNet-18	Yes	Adam	3.06×10^{-3}	1.27×10^{-5}
	No	Adam	3.06×10^{-3}	1.27×10^{-5}

Table 1: Best Hyperparameters for Each Model with and without Data Augmentation

8.3 Results with Data Augmentation

The performance of each model in terms of accuracies,loss, and size of model is shown in the Table 2.

Model	Train Acc.(%)	Test Acc.(%)	Val. Loss	Train Time(s)	Params (M)
Custom CNN	86.71	**86.96**	**0.388**	**337**	**3.09**
ResNet-18	83.96	81.86	0.545	412	11.18
MobileNet	61.17	63.40	1.086	382	1.53
AlexNet	83.92	80.92	0.564	586	57.02

Table 2: Performance and efficiency metrics for models trained with Data Augmentation.

8.3.1 Analysis

- **Custom CNN**: Our custom model performs the best, achieving an accuracy of (86.96%). It shows the strong balance between performance and efficiency.
- **ResNet-18**: This model achieved and accuracy of 81.86% which lower pretty lower than our custom model. ResNet-18 is a complex model and may need more iterations to generalize well.

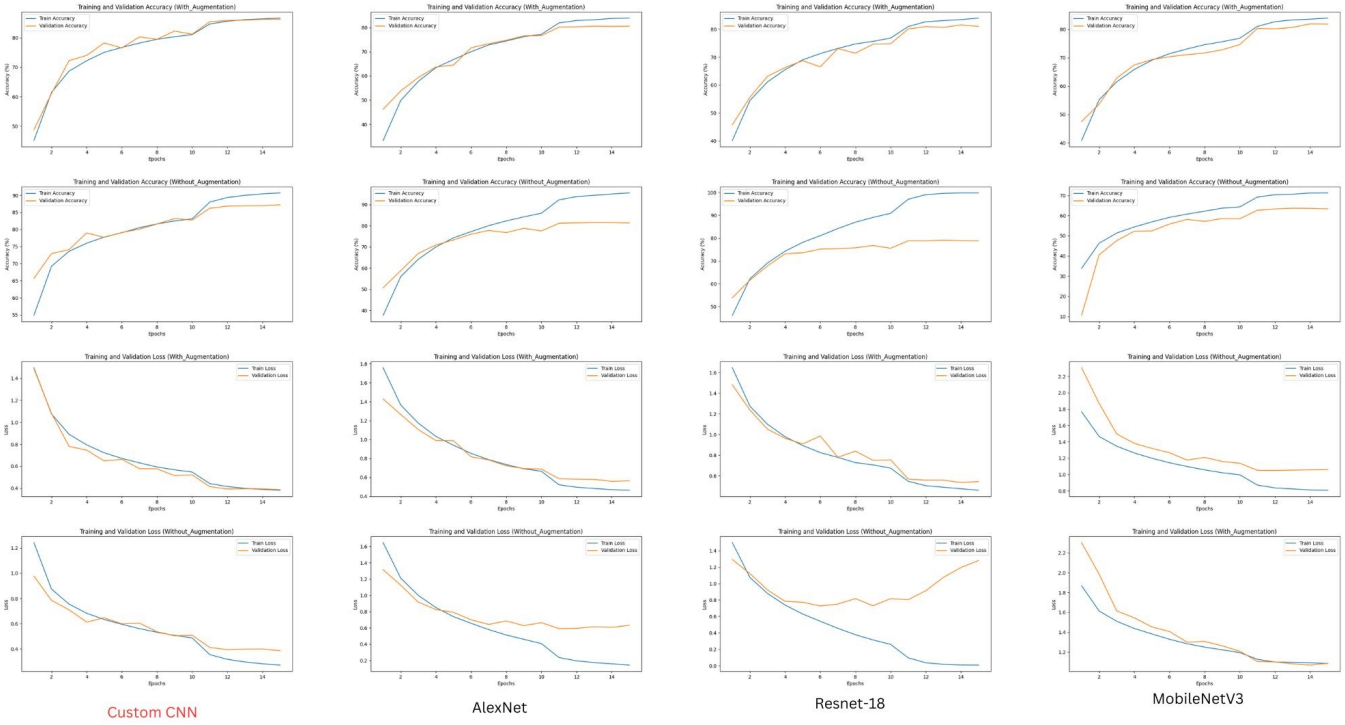


Figure 4: Training and validation loss and accuracy curves with and without data augmentation for each model.

- **MobileNetV3**: It is a lightweight model designed for a constrained environment, it has the lowest accuracy of 63.40% among all models.
- **AlexNet**: Alex net which is the most complex among all, achieved an accuracy of 80.92%. Though it has acceptable accuracy, and is a highly complex model, it does not give any fruitful efficiency.

8.4 Results without Data Augmentation

The performance without data augmentation is shown in Table 3.

Model	Train Acc. (%)	Test Acc. (%)	Val. Loss	Train Time (s)
Custom CNN	87.22	**86.08**	**0.387**	**313**
ResNet-18	99.78	78.12	1.280	444
MobileNetV3	71.22	62.99	1.058	482
AlexNet	95.52	81.60	0.632	462

Table 3: Performance and efficiency metrics for models trained without data augmentation.

8.4.1 Analysis

- **Overfitting**: Since ResNet18 and AlexNet are complex models, they show signs of overfitting due to a lack of enough data in the Cifar-10 dataset. Accuracy of 99.78% and 95.52% has been obtained for these models, whereas their test accuracy is 78.12% and 81.60% respectively, clearly showing the overfitting problem.
- **Custom CNN**: Even without data augmentation, our model gives a significant performance, having a test accuracy of 86.08% which is very close to its training accuracy. It shows that our model can be generalized well for the given task.
- **MobileNetV3**: This model does not perform well, achieving an accuracy of 62.99% on test data which shows that it has a very low impact by using Data Augmentation.

8.5 Comparative Analysis

The results shown in Table 2 confirm that data augmentation improved the model for better generalization. Most importantly, our custom CNN model performed really well on both augmented and non-augmented data. It highlights the effectiveness of its architecture in handling such datasets.

A more detailed analysis of training and loss curves is illustrated in Figure 4.

8.5.1 Observations

- **With Data Augmentation**: In this case, the model showed better generalization i.e. validation accuracy closely followed training accuracyy, which indicated no overfitting.

- **Without Data Augmentation**: In this case, there was significant gap in training accuracy and validation accuracy, especially for ResNet-18 and AlexNet, which confirms overfitting problem.

8.6 Classification Report for the Custom CNN Model

To get better insight from our custom CNN model, a classification chart has been presented in Table 4, which shows the Precision, Recall, and F1-score of each class.

8.6.1 Analysis

Class	Precision	Recall	F1-score
Airplane	0.86	0.87	0.87
Automobile	0.94	0.93	0.93
Bird	0.83	0.79	0.81
Cat	0.74	0.71	0.73
Deer	0.84	0.87	0.85
Dog	0.80	0.79	0.79
Frog	0.87	0.90	0.89
Horse	0.90	0.89	0.90
Ship	0.91	0.92	0.92
Truck	0.90	0.94	0.92

Table 4: Classification report for the custom CNN model with data augmentation.

Table 4 shows the performance of our custom CNN model in all classes in the CIFAR-10 dataset. Most of the classes have good precision and good recall. For classes *automobile*, *ship* and *truck* the values for both precision and recall are above 0.90.

The precision for the class *cat* is comparatively low at 0.74, with a still lower recall of 0.71, a fact which again suggests that cats remains somewhat confused with other animals like dogs and deer. Globally, the accuracy of the model is at 86%.

This consistent performance across classes indeed shows the robustness of our choice of CNN model is doing a reasonably good job handling such a diverse set of categories within the CIFAR-10 dataset.

8.6.2 Analysis of the Confusion Matrix

The confusion matrix in Table 5 gives us an idea, of how well the model differentiates between each class. The key observations obtained from this table are as follows:

- **High Performance on Some Classes**: The model does quite well when classifying various classes such as automobile, ship, and truck with most samples correctly classified. For example: 946 of 1,000 *automobile* were predicted correctly indicating high precision in this class.
- **Confusion between Similar looking classes**: The model often misrecognizes *cat* and *dog*. Among 1,000 *cat* images, the model classified 130 of them as *dog*. This means it has been tough for the model to distinguish between these two classes. The class *truck* is sometimes misclassified as *automobile* since there are 30 instances of this class which probably is because of the similarity in appearance of these kinds of vehicles. Similarly, the *bird* class is also misclassified by other classes as *airplane* with 40 images, *deer* with 37 images and *dog* with 38 images. Here it appears that certain *bird* images have visual ambiguities or resemblance to these particular classes.
- **Strong Results for Other Animal Classes**: The model shows strong performance on classes like *frog* and *horse*, correctly classifying them, with 921 and 906, respectively. However, there is some minor confusion within classes such as *deer* and *dog*.

This confusion matrix helped us to find the strengths and weaknesses of our model. Understanding the patterns of errors will help us to do future improvements in terms of data augmentation or any other additional consideration.

8.7 Discussion

8.7.1 Effect via Data Augmentation

Data augmentation, when compared to without augmentation, turned out to be an important factor in the generalization of models. The models trained on augmented data achieved better test accuracy, and the overfitting effect was negligible when compared with the models trained on original images only.

8.7.2 Custom CNN Performance

The custom CNN model achieved a very good test accuracy when compared with other models and it is also very computationally efficient. The involvement of residual connections, Mish activation functions, and regularization techniques such as batch normalization and dropout makes this architecture very powerful.

8.7.3 ResNet-18 Performance

Although ResNet-18 is a stronger architecture, it takes a longer iteration to fully generalize. In our case, we limited the epochs to 15, and complex models like ResNet-18 require more iteration.

8.7.4 MobileNetV3 Performence

MobileNetV3 showed good balance in performance when since the model is very simple and was developed for an environment with constrained resources. This reduced accuracy indicates that, this model is not suitable for tasks which requires higher precision.

	Airplane	Automobile	Bird	Cat	Deer	Dog	Frog	Horse	Ship	Truck
Airplane	902	9	24	8	5	3	3	5	26	15
Automobile	8	946	0	1	1	1	1	0	7	35
Bird	40	2	808	18	37	38	37	10	6	4
Cat	13	6	40	686	40	130	39	26	11	9
Deer	9	1	42	29	852	13	26	23	4	1
Dog	7	1	15	92	30	813	10	25	5	2
Frog	6	1	18	24	12	11	921	5	1	1
Horse	12	3	7	19	25	21	1	906	0	6
Ship	37	12	0	5	0	1	2	2	935	6
Truck	12	30	2	1	1	2	2	0	23	927

Table 5: Confusion Matrix for the Custom CNN Model with Data Augmentation.

8.8 Conclusion

Conclusion In this work we provided a wide comparison of multiple CNN architectures on the CIFAR-10 dataset. Out of them, the CNN architecture we designed achieved higher test accuracy and also reduced complexity; thus, it can be employed for image classification problems where fewer resources are available. Data Augmentation and hyper-parameter tuning played a critical role in finding the best parameter and improving the performance of each model. In summary, our model provides valuable insights in designing a CNN model, which offers a favorable tradeoff between performance and computational complexity.

8.9 Future Work

Potential directions for future research include:

- **Experimenting with Advanced Architectures**: Experiment with more advanced architecture such as DenseNet, Efficient Net, or the recent Vision Transformer Models on the Cifar-10 Dataset.
- **Extended Hyperparameter Tuning**: By utilizing better computational resources, we can perform wider hyperparameter searches.
- **Application to Larger Datasets**: Applying more complex datasets such as CIFAR-100 on these models to check their scalability and performance in real world situations.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, vol. 25, F. Pereira, C. J. Burges, L. Bottou, and K. Q. Weinberger, Eds., Curran Associates, Inc., 2012.

[2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv preprint arXiv:1409.1556, 2015.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv preprint arXiv:1512.03385, 2015.

[4] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017.

[5] D. Misra, "Mish: A Self Regularized Non-Monotonic Activation Function," arXiv preprint arXiv:1908.08681, 2020.

[6] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," arXiv preprint arXiv:1502.03167, 2015.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[8] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv preprint arXiv:1412.6980, 2017.

[9] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," arXiv preprint arXiv:1907.10902, 2019.

[10] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," arXiv preprint arXiv:1912.01703, 2019.

[11] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.